

Contents

1	Bit Manipulation	2
1.1	Binary number	2
1.2	Bitwise Operator	2
1.3	Bit Masking	3
1.3.1	Get, Set, Clear i -th bit	3
2	Prime Numbers & Factorization	6
2.1	Sieve of Eratosthenes	6
2.2	Segmented Sieve	6
2.3	Primes in range	6
3	String	7
3.1	Sliding Window	7
3.2	Rolling Hash	8

Chapter 1

Bit Manipulation

1.1 Binary number

A binary number is a number expressed in the base-2 numeral system or binary numeral system, it is a method of mathematical expression which uses only two symbols: typically “0” (zero) and “1” (one).

We say that a certain bit is **set**, if it is one, and cleared if it is zero.

1.2 Bitwise Operator

Bitwise AND: Returns 1 only if both the bits are 1.

a = 25	0	0	0	1	1	0	0	1
b = 14	0	0	0	0	1	1	1	0
a & b	0	0	0	0	1	0	0	0

Bitwise OR: Returns 1 if one of the bits is 1.

a = 25	0	0	0	1	1	0	0	1
b = 14	0	0	0	0	1	1	1	0
a b	0	0	0	1	1	1	1	1

Bitwise NOT: Returns 1 if one of the bits is 1.

a = 25	0	0	0	1	1	0	0	1
~ a	1	1	1	0	0	1	1	0

Bitwise XOR: Returns 0 if both the bits are same else 1

a = 25	0	0	0	1	1	0	0	1
b = 14	0	0	0	0	1	1	1	0
a ^ b	0	0	0	1	0	1	1	1

Left Shift: Shifts a towards left by n digits. $a \ll b = a * 2^b$

a = 25	0	0	0	1	1	0	0	1
a << 2	0	1	1	0	0	1	0	0

Right Shift: Shifts a towards right by n digits. $a \gg b = \frac{a}{2^b}$

a = 25	0	0	0	1	1	0	0	1
a >> 2	0	0	0	0	0	1	1	0

1.3 Bit Masking

$x \& 1$: Evaluates to 1 if the number is odd else evaluates to 0.

$x \& (x-1)$: Clears the rightmost set bit of x.

$x \& \sim(x-1)$: Isolates the rightmost set bit of x.

$1 \ll i$: Represent a number with only the i -th bit set.

$\sim(1 \ll i)$: Represent a number with all bits set except the i -th bit.

$n \& (1 \ll i)$: Checks whether i -th bit in n is set or not.

$n \& \sim(1 \ll i)$: Clears the i -th bit in n .

$n \mid (1 \ll i)$: Sets the i -th bit in n without affecting other bits.

$n \mid \sim(1 \ll i)$: Clears all the bits except i -th bit in n .

$n \wedge (1 \ll i)$: Flips the i -th bit in n without affecting other bits.

$n \wedge \sim(1 \ll i)$: Flips the i -th bit and complements all other bits in n .

1.3.1 Get, Set, Clear i -th bit

Get i -th bit

```
int get_ith_bit(int n, int i){
    int mask = (1 << i);
    return (n & mask) > 0 ? 1 : 0;
}
```

Set i -th bit

```
int set_ith_bit(int n, int i){
    int mask = (1 << i);
    return (n|mask);
}
```

Clear i -th bit

```
int clear_ith_bit(int n, int i){
    int mask = ~(1 << i);
    return (n & mask);
}
```

Update i -th bit

```
int update_ith_bit(int n, int i, int v){
    clearIthBit(n,i);
    int mask = (v << i);
    return (n|mask);
}
```

Clear last i bits

```
int clear_last_i_bit(int n, int i){
    int mask = (-1 << i);
    return (n & mask);
}
```

Clear bits in range

```
int clear_bits_in_range(int n, int i, int j){
    int a = -1 << (j+1), b = (1 << i) - 1;
    int mask = (a|b);
    return (n & mask);
}
```

Update bits in range

```
int update_bits_in_range(int n, int i, int v){
    clear_bits_in_range(n,i);
    int mask = (v << i);
    return (n|mask);
}
```

Check if an integer is a power of 2

```
bool isPowerOfTwo(unsigned int n){  
    return !(n & (n-1));  
}
```

Count set bits $O(\text{number of set bits})$

```
int countSetBits(int n){  
    int count = 0;  
    while (n){  
        n = n & (n - 1);  
        count++;  
    }  
    return count;  
}
```

Find the unique number from an array of $2N + 1$ numbers.

Principle : $a \wedge a = 0$ $0 \wedge a = a$

Approach : Take Commulative XOR

Time complexity : $O(N)$

Chapter 2

Prime Numbers & Factorization

2.1 Sieve of Eratosthenes

Sieve of Eratosthenes is an algorithm for finding all the prime numbers $[1 : n]$ using $O(n \log \log n)$ operations.

```
const int N = 10e6;
vector<bool> isPrime(N + 1, true);

void sieve(){
    long long i, j;
    isPrime[0] = isPrime[1] = false;

    for (i = 2; i <= N; i++){
        if (isPrime[i]){
            for (j = i * i; j <= N; j += i)
                isPrime[j] = false;
        }
    }
}
```

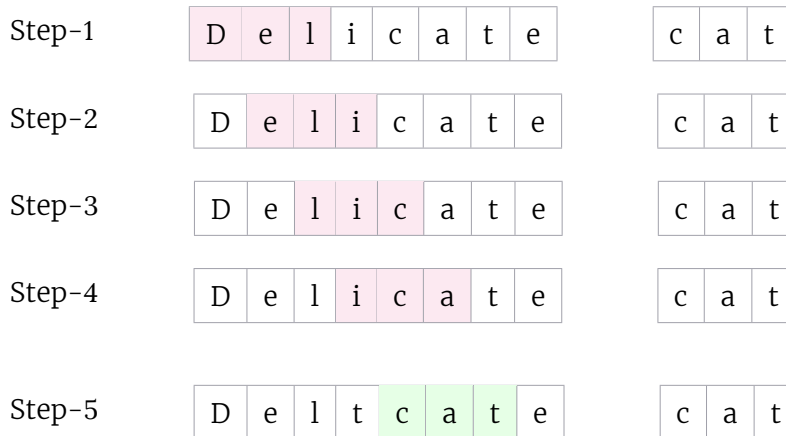
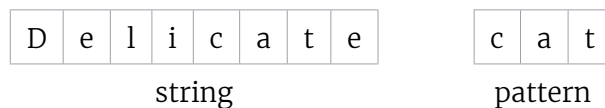
2.2 Segmented Sieve

2.3 Primes in range

Chapter 3

String

3.1 Sliding Window



```
int findIndex(string &str, string &pattern)
{
    int index = -1;
    int n = str.size(), m = pattern.size();

    for (int i = 0; i + m <= n; i++)
        if (str.substr(i, m) == pattern)
            index = i;

    return index;
}
```

3.2 Rolling Hash